

GuardianIQ Step 4

End-to-End Engineering Implementation Specification

Governance Event Architecture

Document Purpose	Developer-ready specification to implement GuardianIQ Step 4 end to end across PostgreSQL, FastAPI/backend services, event processing, React frontend, security, observability, testing, migration and acceptance.
------------------	--

Field	Detail
Prepared for	GuardianIQ Product, Architecture, Engineering, QA and Project Management Teams
Source foundation	GuardianIQ Step 4 - Governance Event Architecture Comprehensive Foundation Document
Build posture	Hybrid architecture: operational state + append-only governance event store
Primary stack assumed	PostgreSQL, FastAPI/Python, React, background dispatcher; broker optional later
Implementation objective	Make every material governance change traceable, correlated, tamper-resistant and reconstructable
Document status	Implementation baseline / developer handoff

Important interpretation note

“Step 4” in this document means the GuardianIQ Governance Event Architecture foundation step. It should not be confused with the “Phase 4” row inside the source foundation’s phase-wise event rollout table. This specification implements the Step 4 architecture itself and integrates it with capabilities already built in earlier GuardianIQ phases.

Contents

1. Executive Implementation Summary
2. Scope, Assumptions and Non-Goals
3. Target Runtime Architecture
4. Implementation Principles and Guardrails
5. Canonical Event Contract
6. PostgreSQL Physical Data Model
7. Database DDL and Migration Strategy
8. Backend Module Architecture
9. Event Publisher Service
10. Transactional Outbox and Dispatcher
11. Event Query, Timeline and Audit Services
12. Event Producer Integration Matrix
13. Event Consumer Processing Matrix
14. REST API Specification
15. Security, Privacy, Integrity and Retention
16. Frontend Information Architecture
17. Frontend Screen Specifications
18. React Component and State Architecture
19. Observability and Operational Monitoring
20. Testing Strategy and Developer Test Matrix
21. Deployment, Migration and Rollback
22. Detailed Build Sequence
23. Definition of Done and Acceptance Criteria
24. Handover Artifacts
- Appendix A. Starter Event Catalogue
- Appendix B. Sample JSON Payloads
- Appendix C. Recommended File Structure
- Appendix D. Engineering Decisions / ADRs

1. Executive Implementation Summary

GuardianIQ Step 4 converts the platform from a collection of governed registries and workflows into an event-first governance system. The implementation must capture every material recommendation, approval, policy decision, escalation, violation, agent action and audit update as a canonical append-only governance event. Operational tables remain the source for current state; the event store becomes the immutable historical truth used for traceability, audit, investigations and governance analytics.

Target outcome	A user must be able to open any governed object or business flow and reconstruct: who/what acted, when, under which policy, with what risk context, which approvals occurred, what action followed, and what evidence was produced.
-----------------------	---

- Create a canonical event schema and stable event taxonomy.
- Create append-only PostgreSQL event storage plus durable transactional outbox.
- Provide a reusable EventPublisherService used by all backend modules.
- Provide search, subject-history, correlation-timeline and audit-export APIs.
- Integrate event production with existing Identity, Registry, Workflow, Relationship, Approval and Agent capabilities.
- Provide frontend Event Explorer, Event Detail, Audit Timeline, Dead-Letter Review and Audit Export screens.
- Enforce tenant isolation, RBAC/ABAC, classification and retention.
- Instrument the pipeline for lag, failures, DLQ volume, high-risk events and event-store growth.
- Deliver executable tests proving immutability, correlation, reliability and audit reconstruction.

2. Scope, Assumptions and Non-Goals

2.1 In Scope

- Canonical event envelope, event types, versions and schemas.
- PostgreSQL event store, outbox, processing log, dead-letter, schema registry, retention and export tables.
- Backend event publisher, validator, repository, dispatcher, timeline, audit export and query services.
- Producer hooks from existing GuardianIQ modules.
- Consumer processing patterns for audit, notification, SLA, dashboard and future integrations.
- Frontend event exploration and auditability experiences.
- Operational security, observability, migration, testing and handover.

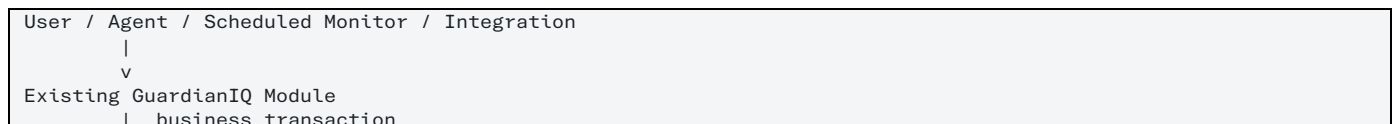
2.2 Assumptions

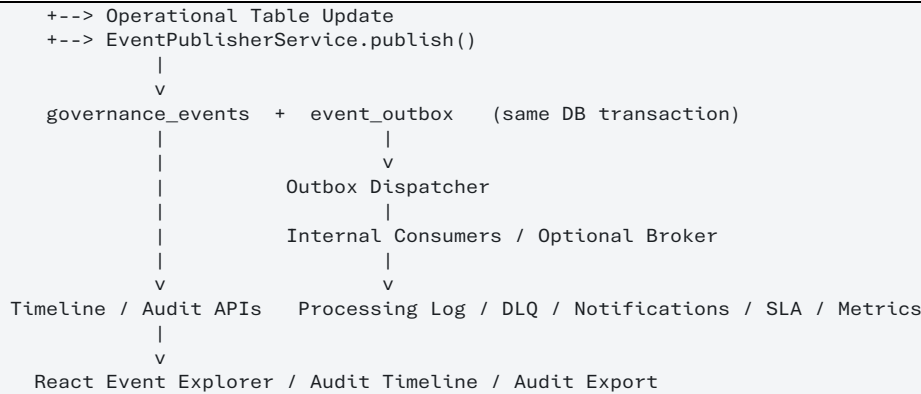
- GuardianIQ already has authentication, tenant context and role/permission foundations from earlier phases.
- PostgreSQL is the system of record for the initial implementation.
- FastAPI/Python is used for backend APIs and services.
- React is used for frontend screens.
- A database-backed outbox + background dispatcher is the first production transport; Kafka/EventBridge/SNS/SQS can be introduced later without changing the canonical event envelope.
- Operational tables continue to exist and are not replaced by full event sourcing.

2.3 Explicit Non-Goals

Non-Goal	Reason
Full event sourcing for every module	Unnecessary complexity for the initial implementation; current state remains in operational tables.
Storing complete documents or model outputs inside events	Use secure artifact/evidence references instead of duplicating large or sensitive content.
Replacing existing audit or workflow modules wholesale	Step 4 integrates and standardizes their event trail.
Introducing a distributed event broker immediately	The architecture must be broker-ready, but database outbox is sufficient initially.
Logging raw secrets or unnecessary PII	Events are governance records, not a data-dump mechanism.

3. Target Runtime Architecture




**Architecture
decision**

Use a hybrid architecture: operational state for fast current-state behavior + append-only governance event history for immutable truth. This matches the Step 4 foundation and avoids premature full event sourcing.

4. Implementation Principles and Guardrails

Principle	Engineering Rule
Append-only truth	Never update or delete an existing governance event through normal application code. Corrections are new events referencing the original.
Atomicity	When a business state change and governance event belong together, write operational state + event + outbox record in one transaction.
Correlation first	Every governed business flow receives a <code>correlation_id</code> at its root and carries it through downstream events.
Past-tense naming	Events describe facts that already occurred: <code>APPROVAL_GRANTED</code> , <code>AGENT_ACTION_BLOCKED</code> .
Small payloads	Events carry governance-relevant facts and references; large evidence and sensitive payloads live in artifact/evidence storage.
Versioned contracts	<code>event_type</code> + <code>event_version</code> identify an immutable schema contract.
Idempotent consumers	Consumers must tolerate at-least-once delivery.
Tenant isolation	<code>tenant_id</code> is mandatory and enforced in every read/write path.
Security by classification	Event visibility and export behavior depend on event classification and user authorization.
Observability by design	Pipeline lag, validation failures, DLQ, high-risk events and processing failures are first-class metrics.

5. Canonical Event Contract

5.1 Canonical Envelope

```

{
  "event_id": "EVT-20260810-000001",
  "event_type": "APPROVAL_GRANTED",
  "event_version": "1.0",
  "tenant_id": "TENANT-001",
  "occurred_at": "2026-08-10T14:15:20Z",
  "recorded_at": "2026-08-10T14:15:21Z",
  "source_service": "approval-service",
  "source_system": "GuardianIQ",
  "actor": {
    "actor_type": "USER",
    "actor_id": "USR-0091",
    "actor_role": "RISK_MANAGER",
    "delegated_from": null
  },
  "subject": {
    "entity_type": "APPROVAL",
    "entity_id": "APR-3001",
    "entity_version": "3"
  },
  "correlation_id": "CORR-REC-1044",
  "causation_id": "EVT-20260810-000000",
  "request_id": "REQ-88991",
  "risk_context": {"risk_score": 86, "risk_level": "HIGH"},
  "policy_context": {"policy_ids": ["POL-007"], "policy_decision": "APPROVAL_REQUIRED"},
  "payload": {"decision": "APPROVED", "decision_rationale": "Controls verified"},
  "integrity": {"hash": "sha256:...", "previous_event_hash": "sha256:..."},

```

```

"classification": "INTERNAL",
"retention_class": "AUDIT_7_YEARS"
}

```

Field	Type	Required	Implementation Rule
event_id	UUID / stable string	Yes	Generated once by server. Never reused.
event_type	varchar(100)	Yes	Must exist in event type registry. Past tense.
event_version	varchar(20)	Yes	Schema version. Consumers branch by version when needed.
tenant_id	UUID/string	Yes	Derived from authenticated tenant context; never trusted from client alone.
occurred_at	timestamptz	Yes	When business fact occurred.
recorded_at	timestamptz	Yes	Server-side insert time.
source_service	varchar(100)	Yes	Logical producer service.
source_system	varchar(100)	Yes	GuardianIQ or trusted external system.
actor	jsonb	Yes	User/Agent/System identity and role.
subject	jsonb	Yes	Primary governed object.
correlation_id	varchar(120)	Yes	End-to-end business lifecycle identifier.
causation_id	uuid/string	Recommended	Immediate parent event/action.
request_id	varchar(120)	Recommended	API/UI request tracing.
risk_context	jsonb	Conditional	Only when risk context is relevant.
policy_context	jsonb	Conditional	Only when policies are evaluated/triggered.
payload	jsonb	Yes	Event-specific schema.
integrity	jsonb	Recommended	Hash / previous hash / signature metadata.
classification	varchar(50)	Yes	PUBLIC / INTERNAL / CONFIDENTIAL / RESTRICTED.
retention_class	varchar(50)	Yes	Mapped to retention rule.

5.2 Event Type and Versioning Rules

- Never change the semantic meaning of an existing event_type + event_version contract.
- Add optional fields only when old consumers remain safe; otherwise create a new event_version.
- Breaking semantic changes require a new version and migration strategy for consumers.
- Event names are uppercase snake case and past tense.
- Every event type must document: category, producer, subject type(s), required payload fields, allowed classifications, retention class and consumers.

6. PostgreSQL Physical Data Model

Table	Purpose	Mutability	Primary Access Pattern
governance_events	Canonical immutable event store	Append only	Search by tenant/type/time/subject/correlation
event_outbox	Reliable dispatch queue	Mutable status only	Pending rows by next_retry_at
event_processing_log	Per-consumer processing result	Append/update processing state	By event_id + consumer_name
event_dead_letter	Failed event processing after retry policy	Controlled admin updates	List/filter/retry
event_schema_registry	Event contract registry	Controlled admin	By event_type + version
event_retention_rules	Retention/archival rules	Controlled admin	By retention_class
event_export_log	Audit export requests/results	Append + controlled status	By requester/scope/date
audit_timeline_read_model	Optional optimized timeline projection	Rebuildable	By tenant/subject/correlation
event_consumer_checkpoint	Optional dispatcher/consumer checkpoint	Mutable	By consumer/partition

6.1 Recommended governance_events Columns

Column	Type	Null?	Notes
event_id	uuid	No	PK; server generated
tenant_id	uuid	No	Tenant isolation
event_type	varchar(100)	No	FK optional to schema registry
event_version	varchar(20)	No	Contract version
occurred_at	timestamptz	No	Business occurrence time
recorded_at	timestamptz	No	default now()
source_service	varchar(100)	No	Producer service
source_system	varchar(100)	No	GuardianIQ/external
actor_json	jsonb	No	Actor envelope
subject_json	jsonb	No	Subject envelope
correlation_id	varchar(120)	No	Business flow
causation_id	uuid	Yes	Parent event if applicable
request_id	varchar(120)	Yes	Request trace
risk_context_json	jsonb	Yes	Risk snapshot

policy_context_json	jsonb	Yes	Policy snapshot
payload_json	jsonb	No	Event payload
classification	varchar(50)	No	Sensitivity class
retention_class	varchar(50)	No	Retention profile
event_hash	varchar(128)	Yes	SHA-256 or equivalent
previous_event_hash	varchar(128)	Yes	Optional chain
created_by_service	varchar(100)	No	Technical identity
inserted_at	timestamptz	No	DB insert timestamp

6.2 Required Indexes

Index	Columns / Expression	Purpose
idx_events_tenant_time	tenant_id, occurred_at DESC	Tenant timeline / recent events
idx_events_type_time	tenant_id, event_type, occurred_at DESC	Event explorer/filtering
idx_events_correlation	tenant_id, correlation_id, occurred_at	Full lifecycle reconstruction
idx_events_subject_gin	GIN(subject_json)	Subject lookup
idx_events_actor_gin	GIN(actor_json)	Actor investigation
idx_events_policy_gin	GIN(policy_context_json)	Policy trigger history
idx_events_payload_gin	GIN(payload_json)	Selective admin/ad-hoc filters; use carefully
idx_outbox_dispatch	status, next_retry_at, created_at	Dispatcher polling
uq_processing_consumer	event_id, consumer_name	Idempotent consumer record

7. Database DDL and Migration Strategy

7.1 Core DDL Skeleton

```

CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE IF NOT EXISTS governance_events (
  event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL,
  event_type VARCHAR(100) NOT NULL,
  event_version VARCHAR(20) NOT NULL,
  occurred_at TIMESTAMPTZ NOT NULL,
  recorded_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  source_service VARCHAR(100) NOT NULL,
  source_system VARCHAR(100) NOT NULL DEFAULT 'GuardianIQ',
  actor_json JSONB NOT NULL,
  subject_json JSONB NOT NULL,
  correlation_id VARCHAR(120) NOT NULL,
  causation_id UUID NULL,
  request_id VARCHAR(120) NULL,
  risk_context_json JSONB NULL,
  policy_context_json JSONB NULL,
  payload_json JSONB NOT NULL DEFAULT '{} '::jsonb,
  classification VARCHAR(50) NOT NULL,
  retention_class VARCHAR(50) NOT NULL,
  event_hash VARCHAR(128) NULL,
  previous_event_hash VARCHAR(128) NULL,
  created_by_service VARCHAR(100) NOT NULL,
  inserted_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS idx_events_tenant_time
  ON governance_events (tenant_id, occurred_at DESC);
CREATE INDEX IF NOT EXISTS idx_events_type_time
  ON governance_events (tenant_id, event_type, occurred_at DESC);
CREATE INDEX IF NOT EXISTS idx_events_correlation
  ON governance_events (tenant_id, correlation_id, occurred_at);
CREATE INDEX IF NOT EXISTS idx_events_subject_gin
  ON governance_events USING GIN (subject_json);
CREATE INDEX IF NOT EXISTS idx_events_actor_gin
  ON governance_events USING GIN (actor_json);
CREATE INDEX IF NOT EXISTS idx_events_policy_gin
  ON governance_events USING GIN (policy_context_json);

```

7.2 Append-Only Enforcement

- Application service account receives INSERT + SELECT only on governance_events; no UPDATE/DELETE.
- Administrative maintenance role is separate and tightly controlled.
- Optional DB trigger rejects UPDATE/DELETE except a designated break-glass maintenance role.
- Corrections are written as EVENT_CORRECTION_RECORDED with original_event_id and correction_reason.
- Partition/archive jobs never mutate event meaning; they only relocate immutable records according to retention policy.

```
CREATE OR REPLACE FUNCTION prevent_event_mutation()
RETURNS trigger AS $$
BEGIN
    RAISE EXCEPTION 'governance_events is append-only';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_governance_events_no_update
BEFORE UPDATE OR DELETE ON governance_events
FOR EACH ROW EXECUTE FUNCTION prevent_event_mutation();
```

7.3 Migration Order

1. Create extensions and reference enums/master tables.
2. Create event_schema_registry and event_retention_rules.
3. Create governance_events and indexes.
4. Create event_outbox, event_processing_log and event_dead_letter.
5. Create event_export_log and optional audit_timeline_read_model.
6. Seed event types, schema versions, classification values and retention classes.
7. Grant least-privilege roles.
8. Run smoke tests before enabling producer hooks.

8. Backend Module Architecture

```
backend/
  modules/
    events/
      event_types.py
      enums.py
      models.py
      schemas.py
      validators.py
      repository.py
      service.py
      integrity.py
      correlation.py
      router.py
      dispatcher.py
      consumer_registry.py
      metrics.py
    audit/
      timeline_service.py
      export_service.py
      router.py
    dlq/
      service.py
      router.py
  shared/
    actor_context.py
    tenant_context.py
    authorization.py
    request_context.py
    security_classification.py
  jobs/
    outbox_dispatch_job.py
    retention_job.py
```

Layer	Responsibility	Must Not Do
Router	HTTP contract, auth dependency, validation handoff	Contain business logic

Service	Business/event orchestration	Write raw SQL directly
Repository	Persistence queries and locking	Make authorization decisions
Validator	Schema and governance validation	Persist side effects
Dispatcher	Reliable outbox delivery and retries	Modify original event
Timeline Service	Transform events into audit-friendly ordered timeline	Invent missing facts
Export Service	Create audit export package and metadata	Bypass classification/permission rules

9. Event Publisher Service

9.1 Service Contract

```
class EventPublisherService:
    def publish(
        self,
        *,
        event_type: EventType,
        subject: SubjectRef,
        payload: dict,
        actor: ActorContext,
        tenant_id: UUID,
        correlation_id: str,
        causation_id: UUID | None = None,
        request_id: str | None = None,
        risk_context: dict | None = None,
        policy_context: dict | None = None,
        classification: EventClassification = EventClassification.INTERNAL,
        retention_class: str = "AUDIT_7_YEARS",
        occurred_at: datetime | None = None,
        db_session: Session | None = None,
    ) -> GovernanceEvent:
        ...
```

9.2 Publish Algorithm

1. Resolve tenant context and actor context from trusted server-side identity/session.
2. Resolve or validate correlation_id; generate one only for root flows.
3. Load event schema for event_type + event_version.
4. Validate payload, subject type, classification and retention class.
5. Generate canonical envelope and integrity hash.
6. Insert governance_events row.
7. Insert event_outbox row in the same database transaction.
8. Return immutable event identifier to producer.
9. Do not directly call downstream consumers from the business transaction.

10. Transactional Outbox and Dispatcher

Column	Purpose
outbox_id	Primary key
event_id	FK/reference to governance event
topic	Logical channel, e.g. guardian.policy
status	PENDING / PROCESSING / DISPATCHED / FAILED / DEAD_LETTER
retry_count	Number of attempts
next_retry_at	Backoff scheduling
locked_by	Worker identifier
locked_at	Lease timestamp
last_error	Most recent failure summary
created_at	Creation time
dispatched_at	Successful dispatch time

Reliability rule

Business state + governance event + outbox row must commit atomically. Delivery may be at-least-once; therefore every consumer must be idempotent.

10.1 Dispatcher Behavior

1. Poll PENDING/FAILED rows whose next_retry_at <= now() using FOR UPDATE SKIP LOCKED.
2. Mark selected row PROCESSING and set worker lease.
3. Load immutable event by event_id.
4. Dispatch to registered internal consumers and/or broker adapter.
5. For each consumer, upsert processing state using event_id + consumer_name as idempotency key.
6. On success mark outbox DISPATCHED.
7. On failure increment retry_count and calculate bounded exponential backoff.
8. After max retries create/update event_dead_letter and mark outbox DEAD_LETTER.
9. Expose metrics for dispatch lag, retry rate and DLQ volume.

11. Event Query, Timeline and Audit Services

Service	Inputs	Output
EventQueryService	tenant + filters	Paged canonical events with authorized fields
SubjectHistoryService	entity_type + entity_id	Chronological events for object
CorrelationTimelineService	correlation_id	End-to-end governed lifecycle
AuditTimelineService	subject/correlation + viewer context	Audit-friendly timeline with actor, policy, risk, decision and evidence
AuditExportService	scope + reason + viewer	Immutable export manifest, event set, hashes and file reference
DLQService	admin filters	Failed consumer processing records and retry action

11.1 Audit Timeline Transformation

- Order primarily by occurred_at, then recorded_at/event_id for deterministic ties.
- Do not hide superseded/correction events; show correction relationship clearly.
- Resolve display names lazily from referenced entities when permitted.
- Flag policy triggers, approvals, overrides, violations, agent blocks and action outcomes visually.
- Preserve raw event JSON access for authorized technical/audit users.
- Never reconstruct or infer facts not present in stored events.

12. Event Producer Integration Matrix

Producer	Trigger	Events	Emit Timing
Identity Service	User created/deactivated; role/delegation changes	USER_CREATED, USER_DEACTIVATED, ROLE_ASSIGNED, ROLE_REVOKED, DELEGATION_STARTED, DELEGATION_ENDED	After successful transaction
AI Model Registry	Create/update/lifecycle	AI_MODEL_REGISTERED, AI_MODEL_UPDATED	Same transaction
AI Agent Registry	Register/change boundary/permission	AGENT_REGISTERED, AGENT_PERMISSION_UPDATED	Same transaction
Tool/Data Source Registry	Register/update	TOOL_REGISTERED, DATA_SOURCE_REGISTERED	Same transaction
Workflow Registry	Register/version/status	WORKFLOW_REGISTERED	Same transaction
Policy Engine	Create/update/activate/evaluate/trigger/retire	POLICY_CREATED, POLICY_UPDATED, POLICY_ACTIVATED, POLICY_EVALUATED, POLICY_TRIGGERED, POLICY_RETIRED	Same transaction or evaluation result
Risk Engine	Risk calculation and threshold changes	RISK_EVALUATED, RISK_THRESHOLD_EXCEEDED, RISK_LEVEL_CHANGED	After scoring
Recommendation Service	Create/enrich/evaluate/withdraw	RECOMMENDATION_CREATED, RECOMMENDATION_ENRICHED, RECOMMENDATION_EVALUATED, RECOMMENDATION_WITHDRAWN	Same transaction
Approval Service	Request/review/grant/reject/change/escalate	APPROVAL_REQUESTED, APPROVAL_REVIEW_STARTED, APPROVAL_GRANTED, APPROVAL_REJECTED, CHANGES_REQUESTED, ESCALATION_TRIGGERED	Same transaction
Action Service	Create/assign/execute/fail/cancel	ACTION_CREATED, ACTION_ASSIGNED, ACTION_EXECUTED, ACTION_FAILED, ACTION_CANCELLED	Same transaction
Agent Runtime	Intent/tool access/validation/block/execute	AGENT_ACTION_REQUESTED, AGENT_ACTION_VALIDATED, AGENT_ACTION_BLOCKED, TOOL_ACCESS_ATTEMPTED, TOOL_ACCESS_DENIED	Before and after execution boundary
SLA Monitor	Warning/breach	SLA_WARNING_RAISED, SLA_BREACHED	Scheduled evaluation
Evidence Service	Evidence/explanation/artifact reference	EVIDENCE_ATTACHED, EXPLANATION_CAPTURED, ARTIFACT_REFERENCED	After secure storage reference created

Audit Service	Timeline view/export/correction	AUDIT_TIMELINE_VIEWED, AUDIT_PACKAGE_EXPORTED, EVENT_CORRECTION_RECORDED	After action completes
---------------	---------------------------------	--	------------------------

13. Event Consumer Processing Matrix

Consumer	Consumes	Side Effect	Idempotency Key	Failure Policy
Audit Projection	All material events	Build/update timeline read model	event_id + consumer	Retry; DLQ
Notification Service	Approval, escalation, SLA, critical violations	Notify role/user/channel	event_id + recipient	Retry; suppress duplicates
SLA Monitor	Approval/action created/status events	Start/stop SLA timers	event_id + SLA rule	Retry
Dashboard Metrics	Risk/policy/agent/approval/action events	Aggregate operational KPIs	event_id + metric projection	Retry/rebuildable
Security Monitor	Unauthorized access, blocked agent actions	Raise security signal	event_id	Immediate alert + retry
Audit Export	Explicit export request	Package event/evidence set	export_id	Controlled retry
External Integration Adapter	Configured event types only	Push to external topic/webhook	event_id + endpoint	Retry + DLQ

14. REST API Specification

Method	Endpoint	Purpose	Minimum Role
POST	/api/v1/events	Publish internal/external governance event	Service/Admin integration role
GET	/api/v1/events	Search events	Authorized user
GET	/api/v1/events/{event_id}	Get one event	Authorized user
GET	/api/v1/events/subject/{entity_type}/{entity_id}	Object event history	Authorized user
GET	/api/v1/events/correlation/{correlation_id}	Full correlation timeline	Authorized user
GET	/api/v1/audit/timeline/{entity_type}/{entity_id}	Audit-friendly timeline	Auditor/Governance
POST	/api/v1/audit/export	Create audit export	Auditor/Admin
GET	/api/v1/audit/export/{export_id}	Export status/metadata	Requester/Admin
GET	/api/v1/events/dead-letter	DLQ list	Platform Admin
POST	/api/v1/events/dead-letter/{id}/retry	Retry DLQ item	Platform Admin
POST	/api/v1/events/replay	Controlled replay of derived consumers	Platform Admin / non-prod preferred

14.1 GET /api/v1/events - Query Parameters

Parameter	Type	Rules
event_type	string/repeatable	Exact event type filter
category	string	Optional category mapping
subject_type	string	Requires subject_id for exact object query if supplied
subject_id	string	Exact subject entity
actor_id	string	Authorization still applied
policy_id	string	Search policy_context
correlation_id	string	Exact flow
classification	string	Cannot request above viewer clearance
occurred_from	datetime	Inclusive
occurred_to	datetime	Exclusive/inclusive by API contract; use consistent convention
page	int	>=1
page_size	int	Default 50, max 200
sort	enum	occurred_at_desc default

14.2 Standard Error Contract

```
{
  "error": {
    "code": "EVENT_SCHEMA_VALIDATION_FAILED",
    "message": "Event payload does not match schema for POLICY_TRIGGERED v1.0",
    "details": [{"field": "payload.policy_id", "issue": "required"}],
    "request_id": "REQ-88991"
  }
}
```

HTTP	Code	Meaning
400	EVENT_SCHEMA_VALIDATION_FAILED	Invalid event contract/payload

401	AUTHENTICATION_REQUIRED	No valid identity
403	EVENT_ACCESS_DENIED	RBAC/ABAC/classification failure
404	EVENT_NOT_FOUND	Unknown event within visible tenant/scope
409	EVENT_IDEMPOTENCY_CONFLICT	Duplicate publish with mismatched content
422	INVALID_QUERY_FILTER	Invalid filter combination
429	RATE_LIMITED	Protection threshold reached
500	EVENT_PERSISTENCE_FAILED	Unexpected server failure

15. Security, Privacy, Integrity and Retention

Control Area	Developer Requirement
Tenant isolation	tenant_id comes from server context; every repository query includes tenant scope.
RBAC/ABAC	Viewer permission evaluated before event query, timeline, export, DLQ and replay.
Classification	PUBLIC / INTERNAL / CONFIDENTIAL / RESTRICTED controls field visibility and export eligibility.
Sensitive payloads	Store references/tokens instead of raw secrets, credentials, full prompts, sensitive documents or unnecessary PII.
Encryption	TLS in transit; database/object storage encryption at rest.
Tamper resistance	Append-only DB permissions + optional trigger + event hashes.
Hashing	Canonicalize selected envelope fields before hashing; document exact algorithm/version.
Retention	retention_class resolves to period, archive tier and delete/anonymize action.
Right-to-erasure/privacy	Keep directly identifying personal data out of immutable payload where possible; reference a separable identity record.
Audit exports	Record requester, reason, scope, event count, hash, generated file reference and expiration.
Replay	Restricted to approved admins; prefer non-production; all replay requests themselves generate audit events.

15.1 Suggested Permission Matrix

Capability	Governance Admin	Auditor	Risk Reviewer	Operator	Read Only
Search normal events	Yes	Yes	Scoped	Scoped	Scoped
View restricted payload	Yes	Conditional	Conditional	No	No
View audit timeline	Yes	Yes	Scoped	Scoped	Scoped
Export audit package	Yes	Yes	No	No	No
View DLQ	Yes	No	No	No	No
Retry DLQ	Yes	No	No	No	No
Replay events	Break-glass	No	No	No	No
View raw JSON	Yes	Yes	Conditional	No	No

16. Frontend Information Architecture

```

Governance
├─ Event Explorer
│   └─ Event Detail Drawer
│       └─ Correlation Timeline
├─ Audit Timeline
├─ Audit Exports
└─ Platform Operations (Admin)
    └─ Dead Letter Queue

Embedded Components
├─ AI Model Details -> Governance Events tab
├─ AI Agent Details -> Governance Events tab
├─ Workflow Details -> Governance Events tab
├─ Relationship Details -> Governance Events tab
└─ Approval/Action Details -> Timeline panel

```

- Event Explorer is an investigation tool, not a generic log viewer.
- Audit Timeline is business-friendly and should prioritize actor, policy, decision, evidence and outcome over raw JSON.
- Raw payload is progressively disclosed in a drawer/tab for authorized users.
- Every object detail page should reuse the same SubjectEventTimeline component rather than implement bespoke histories.
- DLQ and replay controls are admin-only operational tooling.

17. Frontend Screen Specifications

17.1 Event Explorer

Purpose	Provide searchable, filterable visibility into governance events across the user's authorized scope.
----------------	--

Region / Component	Requirement
Header	Title, description, refresh, saved filters (optional later).
Filter bar	Event type/category, actor, subject, policy, classification, date range, correlation ID.
Summary chips	Total results, high-risk count, violations, blocked agent actions when query supports.
Data grid	Occurred time, event type, subject, actor, source, risk, classification, correlation ID.
Row action	Open Event Detail Drawer.
Pagination	Server-side; default 50 rows.

API	Usage
GET /api/v1/events	Primary paginated data source
GET /api/v1/events/{id}	Drawer detail

State	Required UX
Loading	Skeleton rows; preserve filters.
Empty	Explain no events match filters; provide Clear Filters.
Error	Inline error with request_id and retry.
Partial authorization	Never reveal filtered-out restricted counts if not allowed.

User Action	Behavior
Search/Filter	Debounced server query; URL-sync filter state.
Open Event	Drawer opens without losing list state.
Copy correlation ID	Copies identifier; no raw payload exposure.
Navigate subject	Open governed object if user has permission.

- Acceptance: Filters are URL-restorable.
- Acceptance: No cross-tenant data can be surfaced.
- Acceptance: Drawer displays exact event_version and integrity metadata.

17.2 Event Detail Drawer

Purpose	Show the complete canonical event in a readable, permission-aware form.
----------------	---

Region / Component	Requirement
Summary	Event type, timestamp, subject, actor, source, classification.
Governance context	Risk score/level, policies evaluated/triggered, decision.
Payload	Formatted event-specific payload.
Traceability	Correlation ID, causation event link, request ID.
Integrity	Hash and previous hash if present.
Raw JSON	Authorized-only tab.

API	Usage
GET /api/v1/events/{event_id}	Full event detail

State	Required UX
Loading	Drawer skeleton.
Not found	Closeable error; do not leak existence outside scope.
Restricted	Show redacted fields with permission explanation if policy allows metadata visibility.

User Action	Behavior
Open correlation	Navigate to correlation timeline.
Open subject	Navigate to object details.
Copy event ID	Copy safe identifier.

- Acceptance: No editable controls exist.
- Acceptance: Classification controls raw JSON visibility.
- Acceptance: All IDs are clickable only when target is authorized.

17.3 Audit Timeline

Purpose	Reconstruct a governed lifecycle in language understandable to governance, risk, compliance and business reviewers.
----------------	---

Region / Component	Requirement
Context header	Subject/correlation summary, owner, current status when available.
Timeline	Ordered event cards with actor, event, policy/risk badges, rationale, evidence/action links.
Key moments	Visually emphasize approval, rejection, override, violation, agent block and action result.
Export CTA	Visible only to authorized audit roles.
Technical detail toggle	Reveal event IDs, versions and timestamps.

API	Usage
GET /api/v1/audit/timeline/{type}/{id}	Business-friendly timeline
GET /api/v1/events/correlation/{correlation_id}	Full flow when correlation navigation used
POST /api/v1/audit/export	Generate package

State	Required UX
Loading	Progressive skeleton timeline.
Empty	No material governance events recorded.
Error	Retry; preserve context.

User Action	Behavior
Expand event	Show rationale/evidence without leaving timeline.
Export	Open export scope/reason modal.
Open evidence	Navigate/download via evidence service subject to authorization.

- Acceptance: Ordering is deterministic.
- Acceptance: Corrections are visible.
- Acceptance: No derived facts are displayed unless explicitly labeled as computed metadata.

17.4 Dead Letter Queue

Purpose	Allow platform administrators to investigate failed event processing without modifying original events.
---------	---

Region / Component	Requirement
Filters	Consumer, event type, failure date, retry count, status.
Grid	DLQ id, event id, consumer, failure reason, retry count, last attempt.
Detail panel	Original event reference, processing history, error stack summary, safe payload preview.
Retry action	Explicit admin confirmation and reason.

API	Usage
GET /api/v1/events/dead-letter	List
POST /api/v1/events/dead-letter/{id}/retry	Retry

State	Required UX
Loading	Grid skeleton.
Empty	No failed event processing.
Error	Retry and surface request_id.

User Action	Behavior
Retry	Creates audit event and requeues processing; never edits original event.
Open event	Launch Event Detail Drawer.

- Acceptance: Only authorized platform admins can access.
- Acceptance: Retry is idempotent and auditable.
- Acceptance: No raw secrets are rendered.

17.5 Audit Export

Purpose	Create traceable evidence packages for a defined subject, correlation or filtered scope.
---------	--

Region / Component	Requirement
Scope	Subject, correlation or event search filter set.
Reason	Mandatory business/audit reason.
Classification warning	Show maximum classification in scope.
Output options	JSON/CSV/PDF package only if backend supports; otherwise package metadata + secure file reference.
History	Recent exports by requester and status.

API	Usage
POST /api/v1/audit/export	Create export

GET /api/v1/audit/export/{export_id}	Poll status
--------------------------------------	-------------

State	Required UX
Submitting	Disable duplicate requests.
Generating	Show progress/status, not fake percentage.
Completed	Secure link + hash + expiration.
Failed	Reason + retry if permitted.

User Action	Behavior
Create export	Requires reason and confirmation.
Download	Authorized, expiring link.

- Acceptance: Export request itself is audited.
- Acceptance: File hash shown.
- Acceptance: Scope exactly matches request and viewer authorization.

18. React Component and State Architecture

```
src/
  modules/governance-events/
    pages/
      EventExplorerPage.tsx
      AuditTimelinePage.tsx
      DeadLetterQueuePage.tsx
      AuditExportsPage.tsx
    components/
      EventFilterBar.tsx
      EventDataGrid.tsx
      EventDetailDrawer.tsx
      EventTypeBadge.tsx
      RiskBadge.tsx
      ClassificationBadge.tsx
      CorrelationTimeline.tsx
      SubjectEventTimeline.tsx
      AuditExportDialog.tsx
      DlqRetryDialog.tsx
    api/
      governanceEventsApi.ts
    hooks/
      useEventsQuery.ts
      useEventDetail.ts
      useAuditTimeline.ts
    types/
      governanceEvent.ts
    utils/
      eventDisplay.ts
      timelineGrouping.ts
```

Concern	Recommendation
Server state	Use existing query library pattern (e.g. React Query) for caching, cancellation and retry; do not duplicate server state in global store.
URL state	Persist filters, page, sort and selected correlation in query params where useful.
Global state	Only auth/tenant/layout/shared app state.
Type safety	Generate or hand-maintain TypeScript contracts aligned with OpenAPI schemas.
JSON rendering	Safe syntax viewer; never dangerouslySetInnerHTML.
Large payloads	Lazy-load raw JSON section; backend should cap payload size.
Accessibility	Keyboard-operable filters, grids, drawers and timeline; color is not the only status indicator.
Responsive	Target enterprise laptop/desktop first; grid collapses to key columns on narrow screens.

19. Observability and Operational Monitoring

Metric	Type	Labels / Notes
events_published_total	counter	event_type, source_service, status
event_validation_failures_total	counter	event_type, source_service, reason
event_outbox_pending	gauge	topic
event_outbox_dispatch_lag_seconds	gauge/histogram	topic
event_dispatch_failures_total	counter	topic, consumer
event_dlq_total	gauge/counter	consumer, event_type

event_consumer_latency_seconds	histogram	consumer
high_risk_events_total	counter	event_type, policy/risk level
agent_actions_blocked_total	counter	agent/tool/policy (careful cardinality)
audit_export_failures_total	counter	reason
governance_event_store_rows	gauge	tenant only if cardinality manageable

Alert	Trigger	Owner	Response
Event pipeline stalled	Oldest outbox PENDING age > threshold	Platform Engineering	Inspect dispatcher DB locks, worker health, DB connectivity
DLQ spike	DLQ count/rate above baseline	Platform Engineering	Inspect failing consumer/schema mismatch
Critical policy violation spike	POLICY_VIOLATION_DETECTED above baseline	Governance/Risk	Investigate correlated objects/actors
Blocked critical agent action	AGENT_ACTION_BLOCKED on high-criticality tool	AI Governance Owner	Review policy and attempted action
Audit export failures	Export error rate above threshold	Platform + Compliance	Check storage/permissions/schema
Event store growth anomaly	Unexpected daily growth	Platform Engineering	Review noisy producers / retention

20. Testing Strategy and Developer Test Matrix

Test Layer	Required Coverage
Unit - schemas	Required fields, enums, versions, payload constraints, classification/retention validation.
Unit - publisher	Envelope enrichment, correlation, actor context, hash generation, outbox insertion.
Repository	Tenant filters, index-supported queries, append-only behavior, pagination.
Transactional	Operational update + event + outbox commit/rollback atomicity.
Dispatcher	Locking, concurrent workers, retry/backoff, DLQ, idempotent delivery.
Consumer	Duplicate delivery, partial failure, replay behavior.
Security	Cross-tenant denial, role denial, classification redaction, export/replay admin restrictions.
Timeline	Ordering, correlation, causation, corrections and missing optional context.
Frontend	Loading/empty/error states, filters, timeline, raw JSON access, DLQ retry.
Performance	Event search, subject history and correlation timeline under expected volume.
Recovery	Worker crash after processing/before ack, DB outage, duplicate outbox dispatch.

20.1 Mandatory End-to-End Scenarios

ID	Scenario	Expected Result
E2E-01	Recommendation -> Policy -> Risk -> Approval -> Action -> Audit	All events share correlation_id; timeline complete and ordered.
E2E-02	Agent attempts restricted ERP update	AGENT_ACTION_REQUESTED -> TOOL_ACCESS_ATTEMPTED -> POLICY_EVALUATED -> AGENT_ACTION_BLOCKED -> POLICY_VIOLATION_DETECTED -> audit visible.
E2E-03	Approval rejected	Rejection prevents action execution; timeline shows reviewer/rationale.
E2E-04	SLA breach	Approval/action stays pending past threshold; SLA_BREACHED emitted and visible.
E2E-05	Consumer failure	Dispatcher retries; item reaches DLQ; admin retry succeeds; original event unchanged.
E2E-06	Correction	Incorrect event followed by EVENT_CORRECTION_RECORDED; both visible.
E2E-07	Cross-tenant read attempt	API returns denied/not found per security convention; no leakage.
E2E-08	Audit export	Authorized user generates package with scope, hash, requester and export log.

21. Deployment, Migration and Rollback

21.1 Release Sequence

1. Deploy backward-compatible database migrations with no producer hooks enabled.
2. Deploy backend event module, query APIs and dispatcher disabled by feature flag.
3. Seed schema registry, event types, retention rules and classifications.
4. Run synthetic publish/query/dispatch smoke tests.
5. Enable publisher for one low-risk producer (e.g. model registry) in dev/test.
6. Enable dispatcher and verify processing logs/DLQ.
7. Enable remaining producer hooks in controlled groups.
8. Deploy frontend Event Explorer and Audit Timeline.
9. Enable audit export and admin DLQ tooling last.
10. Observe metrics before broad production enablement.

21.2 Rollback Rules

- Never roll back by deleting already-recorded governance events.

- Producer hooks can be disabled via feature flags while retaining event store data.
- Dispatcher can be paused safely; outbox preserves pending events.
- Frontend can be rolled back independently if API contracts remain compatible.
- Database migrations should be additive for the first Step 4 release; destructive migrations require a separate controlled release.

22. Detailed Build Sequence

Build Stage	Required Output
Step 1 - Contract Lock	Freeze event envelope, taxonomy MVP, versions, classifications, retention classes and producer/consumer list.
Step 2 - Database Foundation	Implement tables, indexes, grants, append-only protection, seed/reference data and migrations.
Step 3 - Backend Core	Implement Pydantic schemas, validators, repository, publisher service, correlation and actor context.
Step 4 - Reliability	Implement outbox, dispatcher, processing log, retry/backoff, DLQ and idempotency.
Step 5 - Query and Audit	Implement event search, subject history, correlation timeline, audit timeline and export services.
Step 6 - Producer Integration	Add hooks to registry, policy, risk, recommendation, approval, action, agent and evidence modules.
Step 7 - Frontend	Implement Event Explorer, Event Detail, Audit Timeline, DLQ and Audit Export plus embedded object timeline.
Step 8 - Security and Observability	Apply RBAC/ABAC/classification, metrics, alerts, rate limits and retention controls.
Step 9 - QA and Recovery	Execute unit, integration, security, concurrency, failure and E2E scenarios.
Step 10 - Production Enablement	Feature-flag rollout, monitoring, handover and acceptance sign-off.

23. Definition of Done and Acceptance Criteria

- AC-01** Every major state-changing operation selected for the Step 4 release emits a canonical governance event.
- AC-02** Events validate against event_type + event_version schema before persistence.
- AC-03** governance_events is append-only for normal application roles.
- AC-04** Operational state + event + outbox are atomic for integrated business transactions.
- AC-05** Every event includes tenant, actor, subject, occurred_at, source_service and correlation_id.
- AC-06** At least one complete governed lifecycle reconstructs end to end in the UI.
- AC-07** Event APIs support query by type, subject, actor, date, policy and correlation.
- AC-08** Outbox processing is durable, retryable and observable.
- AC-09** Consumer processing is idempotent and failures reach a reviewable DLQ.
- AC-10** Event visibility respects tenant, RBAC/ABAC and classification.
- AC-11** Audit Timeline is business-readable and raw JSON remains available to authorized users.
- AC-12** Audit export logs requester, reason, scope, generated artifact and integrity hash.
- AC-13** Critical metrics and alerts are active.
- AC-14** Automated test suite covers canonical E2E scenarios and recovery conditions.
- AC-15** No critical or high-severity defects remain open at handover.

24. Handover Artifacts

Artifact	Minimum Content	Owner
Database Migration Pack	DDL, indexes, grants, seeds, append-only protection, rollback notes	Backend/DB
Event Contract Catalogue	Event type, version, schema, producer, subject, classification, retention	Backend/Architecture
OpenAPI Specification	All event/audit/DLQ APIs with schemas and errors	Backend
Producer Integration Matrix	Source operation -> emitted event -> transaction timing	Backend
Consumer Matrix	Consumer -> side effect -> idempotency -> retry/DLQ behavior	Backend
Frontend Route/Component Map	Pages, routes, reusable timeline/drawer components	Frontend
QA Evidence Pack	Automated + manual scenario results, screenshots/logs where applicable	QA
Operational Runbook	Metrics, alerts, DLQ handling, dispatcher restart, export failure handling	Platform/QA
Security Review Checklist	Tenant, role, classification, secrets, export/replay controls	Security/Backend
Known Limitations / Backlog	Deferred broker, replay depth, archival automation, advanced analytics	PM/Architecture

Appendix A. Starter Event Catalogue

Category	Event Type	Description
Identity	USER_CREATED	User profile created
Identity	USER_DEACTIVATED	User access disabled
Identity	ROLE_ASSIGNED	Role granted
Identity	ROLE_REVOKED	Role removed
Identity	DELEGATION_STARTED	Delegated authority starts
Identity	DELEGATION_ENDED	Delegated authority ends
Registry	AI_MODEL_REGISTERED	AI model registered
Registry	AI_MODEL_UPDATED	AI model metadata updated
Registry	AGENT_REGISTERED	AI agent registered
Registry	AGENT_PERMISSION_UPDATED	Agent permission changed
Registry	TOOL_REGISTERED	Tool/API connector registered
Registry	DATA_SOURCE_REGISTERED	Data source registered
Registry	WORKFLOW_REGISTERED	Workflow registered
Policy	POLICY_CREATED	Policy created
Policy	POLICY_UPDATED	Policy updated
Policy	POLICY_ACTIVATED	Policy activated
Policy	POLICY_RETIRED	Policy retired
Policy	POLICY_EVALUATED	Policy evaluation completed
Policy	POLICY_TRIGGERED	Policy condition triggered
Risk	RISK_EVALUATED	Risk score calculated
Risk	RISK_THRESHOLD_EXCEEDED	Threshold exceeded
Risk	RISK_LEVEL_CHANGED	Risk level changed
Recommendation	RECOMMENDATION_CREATED	Recommendation created
Recommendation	RECOMMENDATION_ENRICHED	Context/evidence added
Recommendation	RECOMMENDATION_EVALUATED	Recommendation evaluated
Recommendation	RECOMMENDATION_WITHDRAWN	Recommendation withdrawn
Approval	APPROVAL_REQUESTED	Approval requested
Approval	APPROVAL_REVIEW_STARTED	Review started
Approval	APPROVAL_GRANTED	Approval granted
Approval	APPROVAL_REJECTED	Approval rejected
Approval	CHANGES_REQUESTED	Changes requested
Escalation	ESCALATION_TRIGGERED	Escalation created
Escalation	ESCALATION_ASSIGNED	Escalation assigned
Escalation	ESCALATION_RESOLVED	Escalation resolved
Action	ACTION_CREATED	Action created
Action	ACTION_ASSIGNED	Action assigned
Action	ACTION_EXECUTED	Action executed
Action	ACTION_FAILED	Action failed
Action	ACTION_CANCELLED	Action cancelled
Agent	AGENT_ACTION_REQUESTED	Agent requested action
Agent	AGENT_ACTION_VALIDATED	Action validated
Agent	AGENT_ACTION_BLOCKED	Action blocked
Agent	TOOL_ACCESS_ATTEMPTED	Tool access attempted
Agent	TOOL_ACCESS_DENIED	Tool access denied
Violation	POLICY_VIOLATION_DETECTED	Policy violation detected
Violation	SLA_WARNING_RAISED	SLA warning
Violation	SLA_BREACHED	SLA breached
Violation	UNAUTHORIZED_ACCESS_ATTEMPTED	Unauthorized attempt
Evidence	EVIDENCE_ATTACHED	Evidence linked
Evidence	EXPLANATION_CAPTURED	Explanation captured
Evidence	ARTIFACT_REFERENCED	Artifact reference added
Audit	AUDIT_TIMELINE_VIEWED	Timeline viewed
Audit	AUDIT_PACKAGE_EXPORTED	Audit package exported
Audit	EVENT_CORRECTION_RECORDED	Correction recorded

Appendix B. Sample JSON Payloads

B.1 POLICY_TRIGGERED

```
{
  "policy_id": "POL-COMM-010",
  "policy_name": "High Value Quote Approval Policy",
  "trigger_condition": "quote_value > 500000 OR risk_score >= 80",
  "decision": "APPROVAL_REQUIRED",
  "required_approver_role": "COMMERCIAL_HEAD",
  "sla_hours": 8
}
```

B.2 APPROVAL_GRANTED

```
{
  "approval_id": "APR-3001",
  "decision": "APPROVED",
  "decision_rationale": "Risk reviewed and controls confirmed",
  "reviewer_comments": "Proceed with human-supervised execution",
  "sla_status": "WITHIN_SLA"
}
```

B.3 AGENT_ACTION_BLOCKED

```
{
  "agent_id": "AGT-001",
  "tool_id": "ERP-PRICE-API",
  "requested_operation": "UPDATE_PRICE",
  "target_reference": "PRODUCT-1001",
  "block_reason": "Human approval required for price changes",
  "policy_id": "POL-AGENT-PRICE-001",
  "required_next_step": "APPROVAL_REQUESTED"
}
```

B.4 EVENT_CORRECTION_RECORDED

```
{
  "original_event_id": "EVT-000019",
  "correction_reason": "Incorrect risk score metadata received from upstream calculation",
  "corrected_fields": {"risk_score": 86},
  "authorized_by": "USR-ADMIN-01"
}
```

Appendix C. Recommended File Structure

```
backend/
  modules/events/
  modules/audit/
  modules/dlq/
  shared/actor_context.py
  shared/tenant_context.py
  shared/authorization.py
  jobs/outbox_dispatch_job.py
  migrations/versions/<step4_event_architecture>.py

tests/
  unit/events/
  integration/events/
  e2e/governance_timeline/

frontend/src/modules/governance-events/
  pages/
  components/
  api/
  hooks/
  types/
  utils/
```

Appendix D. Engineering Decisions / ADRs

ADR	Decision	Rationale
ADR-001	Hybrid operational state + append-only events	Practical auditability without full event-sourcing complexity.
ADR-002	Database transactional outbox first	Reliable initial architecture; broker can be added later.
ADR-003	Canonical envelope shared across modules	Stable traceability and consistent analytics.
ADR-004	At-least-once delivery + idempotent consumers	Simpler reliable processing than exactly-once claims.
ADR-005	Small payloads + artifact references	Protect privacy, reduce event-store bloat and support retention.

ADR-006	Server-derived tenant and actor context	Prevent spoofed accountability metadata.
ADR-007	Append-only correction events	Preserve forensic truth and chain of custody.
ADR-008	Business timeline separated from raw event JSON	Reader-friendly audit experience with technical drill-down.

Source and Alignment Note

This implementation specification is grounded in the uploaded “GuardianIQ - Step 4 Governance Event Architecture - Comprehensive Foundation Document.” It preserves that foundation’s core decisions: event-first governance, canonical event envelope, append-only event logging, operational tables plus event history, durable outbox, event correlation, event store/audit store separation, security/retention controls, developer implementation guidance, testing and acceptance. The additional database constraints, API contract details, frontend screen behavior, outbox concurrency/idempotency rules, deployment steps and handover artifacts in this document are engineering elaborations intended to make the foundation directly implementable by the GuardianIQ development team.